



ALBUKHARY INTERNATIONAL UNIVERSITY

MACHINE LEARNING REPORT

Mushroom Classification: Predicting Edibility Using Machine Learning

PROJECT TITLE	Mushroom Classification: Predicting Edibility Using Machine Learning
COURSE NAME	MACHINE LEARNING
LECTURER:	PROF DR ZURINAHNI ZAINOL
STUDENT NAME	MARALBYEK TILYEK
MATRIC NUMBER	AIU24102280
DATASET	MUSHROOM DATA SET T11
SUBMISSION DATE	APRIL 29, 2026, 5:00PM

maralbyek.tilyek@student.aiu.edu.my

Apr 29, 2026

INDIVIDUAL MACHINE LEARNING.....	0
ASSIGNMENT REPORT.....	0
ABSTRACT.....	3
3. INTRODUCTION.....	4
3.1 Background.....	4
3.2 Problem Statement.....	4
3.3 Objectives.....	4
4. DATASET DESCRIPTION.....	5
4.1 Dataset Source.....	5
4.2 Dataset Overview.....	5
4.3 Feature Description Table.....	5
5. EXPLORATORY DATA ANALYSIS (EDA).....	7
5.1 Data Understanding.....	7
5.2 Data Distribution.....	7
5.2a Feature Visualization.....	7
5.3 Relationships.....	8
5.4 Key Findings.....	8
6. DATA PREPROCESSING.....	9
6.1 Handling Missing Values.....	9
6.2 Encoding Categorical Variables.....	9
6.3 Feature Scaling.....	10
6.4 Outlier Handling.....	11
6.5 Handling Imbalanced Data.....	11
6.6 Final Dataset.....	12
7. DIMENSION REDUCTION.....	13
7.1 Method Used.....	13
7.2 Justification.....	13
7.3 Results.....	13
7.4 Decision: Retain All 22 Features.....	14
8. SUPERVISED LEARNING MODELS.....	15
8.1 Model 1: Logistic Regression.....	15
8.2 Model 2: Logistic Regression.....	15
Algorithm Overview:.....	15
Mathematical Foundation:.....	15
Advantages:.....	16
Limitations:.....	16
Reason for Choice:.....	16
9. MODEL EVALUATION.....	17

9.1 Evaluation Methodology.....	17
9.2 Performance Metrics.....	17
9.3 Results - Logistic Regression.....	17
9.4 Results - Decision tree.....	18
9.5 Model Comparison Table.....	18
9.6 Best Model Justification.....	18
10. DISCUSSION.....	19
10.1 Model Comparison Analysis.....	19
10.2 Feature Importance Analysis.....	19
10.3 Impact of Preprocessing.....	20
11. CHALLENGES.....	21
12. CONCLUSION.....	22
13. REFERENCES (2018-2026).....	23
14. APPENDIX.....	24
14.1. Jupyter Notebook Code Structure.....	24

ABSTRACT

Problem: The proper identification of mushrooms and the safety of consumption is a vital aspect in maintaining the food chain and in ecological studies. Failure to distinguish toxic mushrooms will result in health consequences. The present project focuses on the binary classification problem where mushrooms are either classified as toxic or non-toxic using morphological data.

Dataset: The analysis makes use of the UCI Mushroom Database which contains 8,124 instances, each with 22 categorical variables, describing the morphological properties (such as cap, gill, stem, and spore print properties). For demonstrating practical data handling skills, the data set, which originally had 100% completeness, has been increased by adding **5% artificial missing values (~405 data points)** in five important attributes, namely odor, gill size, gill attachment, gill color, and stalk shape. The Mode Imputation method is used to recover the 100% completeness. Including **60 outliers** added in order to satisfy the outlier handling function and similar case goes for imbalance of the dataset by making the dataset 70/30 percentages of imbalance

Models: The two supervised classification techniques applied include: (i) logistic regression for the linear classifier and (ii) decision tree as a non-linear classifier. These two methods were assessed through stratified fivefold cross-validation using an 80% training set and a 20% test set, with the F1-score being the main measure of performance.

Key Results: Decision tree outperformed Logistic Regression with higher performance metrics such as an accuracy of 99.6% and an F1 score of 0.9969 against Logistic Regression's accuracy of 96.0% and an F1 score of 0.9596, thus proving that the edibility of mushrooms is based on complex non-linear relationships between its features.

3. INTRODUCTION

3.1 Background

Mushroom identification and safety evaluation is a very important field in ecology, food science, and medicine. The identification error of poisonous mushrooms may cause serious health issues, such as organ failure and even death. In this case, machine learning-based algorithms can play an extremely valuable role in improving the safety of foods and ecological research. The mushroom data set is one of the oldest and well-known test data sets used in machine learning applications. It is commonly applied to develop classification algorithms and evaluate their efficiency. In this context, the creation of automated classification systems based on machine learning has a tremendous potential to improve the safety of foods, conduct ecological research, and spread mushroom identification skills globally. Machine learning methods allow us to classify mushrooms quickly and objectively by analyzing different morphological characteristics at once without any specialized knowledge. The mushroom data set is one of the oldest and well-known test data sets used in machine learning applications. It is commonly applied to develop classification algorithms and evaluate their efficiency. Its frequent application in numerous scientific works throughout decades makes it an appropriate basis for testing classification techniques.

3.2 Problem Statement

Based on the morphology and ecology of mushrooms, predict whether a mushroom is poisonous or edible. This entails working with a multitude of categorical data fields whose distributions differ and selecting an appropriate classification algorithm that provides high accuracy. This will involve a binary classification model whose performance should be evaluated through various metrics.

3.3 Objectives

- To perform a thorough exploratory data analysis on the mushroom dataset
-
- To utilize proper preprocessing techniques for categorical variables without adding any bias
-
- To analyze the performance of different supervised learning models through a proper methodology
-
- To determine important features that affect edible mushrooms
-
- To make practical suggestions for classification of mushrooms

4. DATASET DESCRIPTION

4.1 Dataset Source

Dataset Source: UCI Machine Learning Repository (<https://www.cs.toronto.edu/~delve/data/mushrooms/mushroomDetail.html>). Original source: Mushroom Records Database, contributed by Jeff Schlimmer. Updated: April 24, 2018.

4.2 Dataset Overview

Attribute	Value
Total Samples	8,124 mushroom specimens
Total Features	22 categorical attributes
Target Variable	Class (Edible/Poisonous)
Feature Type	All categorical (nominal)
Missing Values	Missing Values 406 (5%) 0 (100% complete) Mode Imputation
Class Distribution	Edible: 4,208 (51.80%), Poisonous: 3,916 (48.20%)
Data Type	CSV format
Balance Status	Majority : Minority = 2.5 : 1 % = 71.4% : 28.6%

4.3 Feature Description Table

Feature	Type	Values	Description
Cap-shape	Categorical	b, c, x, f, k, s	Shape of mushroom cap
Cap-surface	Categorical	f, g, y, s	Surface texture of cap
Cap-color	Categorical	n, b, c, g, r, p, u, e, w, y	Color of mushroom cap
Bruises	Binary	t, f	Presence of bruises
Odor	Categorical	a, l, c, y, f, m, n, p, s	Smell characteristics
Gill-attachment	Categorical	a, d, f, n	How gills attach to stalk

Gill-spacing	Categorical	c, w, d	Distance between gills
Gill-size	Categorical	b, n	Size of gill structure
Gill-color	Categorical	k, n, b, h, g, r, o, p, u, e, w, y	Color of gills
Stalk-shape	Categorical	e, t	Shape of mushroom stalk
Stalk-root	Categorical	b, c, u, e, z, r	Root structure beneath ground
Stalk-surface-above-ring	Categorical	f, y, k, s	Stalk texture above ring
Stalk-surface-below-ring	Categorical	f, y, k, s	Stalk texture below ring
Stalk-color-above-ring	Categorical	n, b, c, g, o, p, e, w, y	Color above gill attachment
Stalk-color-below-ring	Categorical	n, b, c, g, o, p, e, w, y	Color below gill attachment
Veil-type	Categorical	p, u	Type of veil structure
Veil-color	Categorical	n, o, w, y	Veil coloration
Ring-number	Categorical	n, o, t	Ring quantity on stalk
Ring-type	Categorical	c, e, f, l, n, p, s, z	Ring shape and attachment
Spore-print-color	Categorical	k, n, b, h, r, o, u, w, y	Color of spore print
Population	Categorical	a, c, n, s, v, y	Typical population amount
Habitat	Categorical	g, l, m, p, u, w, d	Common growing environment

5. EXPLORATORY DATA ANALYSIS (EDA)

5.1 Data Understanding

Mushroom Data Set is an established data set that is well documented and has complete feature coverage. The initial dataset had no missing values (**100% complete**). In order to illustrate the technique of data preprocessing and missing value handling, **5% missing values (405 instances)** have been added to five attributes: odor, gill-size, gill-attachment, gill-color, and stalk-shape. **Mode Imputation** has then been performed to make the dataset again 100% complete. All the attributes are categorical and represent different morphological attributes such as shape, texture, and color of cap; gill attachment, spacing, size, and color; shape, root, texture, and color of stalk; and habitat and population.

Moreover, imbalance and outliers were found as outliers being **60** and imbalance ratio being **2.5 : 1**.

5.2 Data Distribution

Class Distribution Analysis:

Class	Count	Percentage	Samples
Edible	4,208	51.80%	Majority class
Poisonous	3,916	48.20%	Minority class
Total	8,124	100%	Complete

Figure 1a & b: Class distribution (Count & Percentage)



Class Balance Check: This dataset has excellent class balance in terms of imbalance ratio which is **1.05:1**. Class balance of this nature means that no oversampling, undersampling, or SMOTE methods will be required. Both the classes are well-balanced, providing enough instances for the model to learn from. But handled through **SimpleImputer** from **2.5 : 1** after adding it purposefully.

5.2a Feature Visualization

The analysis examines the 6 most important features for edibility classification:

- **Odor** (9 categories)
- **Gill-Size** (2 categories)
- **Gill-Attachment** (4 categories)
- **Gill-Color** (12 categories)
- **Stalk-Shape** (2 categories)
- **Cap-Shape** (6 categories)

Visualization Method:

Graphs with coloring based on whether the mushrooms are edible or poisonous in hues of green (edible) and red (poisonous). This kind of plot demonstrates clear differentiation trends among characteristics such as smell and gill type for edible and poisonous mushrooms.

Key Observations from Plots:

Odor shows strongest class separation (pungent odors almost exclusively poisonous)

Gill-size has distinct distribution between classes

Cap-shape less discriminative than gill/odor features

5.3 Relationships

A preliminary analysis indicates that there is a very strong correlation between the smell attributes and the edibility of the mushrooms. The mushrooms having strong or almond smells have significantly different distributions in terms of their edibility as compared to those without any smell at all. The attributes of gills such as size, attachment, and separation also have a strong correlation with the class labels.

5.4 Key Findings

Dealing with data completeness: The original data had a 100 percent completeness rate (no missing values). With the creation of artificial missing values at a **5 percent rate (405 cases)**, Mode Imputation was used to achieve full data completeness (8,124 cases, missing value ratio = 0%).

- Class balance is achieved: 51.8% edible against 48.2% poisonous mushrooms.
- Categorical attributes with 22 attributes covering all morphological aspects.
- No duplicate data ensures data accuracy.
- Wide-ranging attributes may imply non-linear relations, which can be used for classification.
- Attribute Odor becomes an important classifier for mushrooms.
- 60 Outliers successfully found
- Imbalancement of 2.5 : 1 were also found in the dataset

6. DATA PREPROCESSING

6.1 Handling Missing Values

Original Data Set: 100% complete (with no missing values)

Added Artificially: 5% missing values (approximately 405 observations) added to Odor, Gill-size, Gill-attachment, Gill-color, Stalk-shape on the dataset by adding missing values on purpose because of the original dataset being completely perfect.

All the 405 values are random in the dataset mixed with different attributes

Imputation will add the most frequent value of the attribute that is mentioned in the dataset. E.g. if the **0.8/1** of the dataset is **n** then the value will be **n** since the frequently mentioned attribute value is **n**

Purpose: To show how missing data can be handled from reality

Missing Value Imputation Strategy: Mode Imputation (using most frequent category)

Implementation:

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='most_frequent')
X_filled = imputer.fit_transform(X_with_missing)
```

In order to handle the missing value, Missing Value Detection was used which belongs to Pandas, specifically **.isnull()** and **.sum()**.

6.2 Encoding Categorical Variables

Technique Used: Label Encoding

Since the mushroom dataset consists entirely of categorical attributes represented by letter codes (such as **p**, **e**, **f**, **n**), Label Encoding was applied to convert these values into numerical form so that machine learning algorithms could process them effectively.

Each unique category in every feature was assigned a unique integer value. For example, values such as **odor = p**, **odor = a**, and **odor = f** were transformed into numeric labels like **6**, **0**, and **2** respectively.

Before encoding, missing values were first handled using **Mode Imputation (Most Frequent Value)** to replace all **NaN** values with the most common category in each affected feature. After ensuring the dataset contained no missing values, Label Encoding was performed on all columns.

Rationale for Choosing Label Encoding

Simple and Efficient: Converts categorical values into numeric form using one-to-one mapping without losing information.

Memory Efficient: Requires less memory and faster processing compared to One-Hot Encoding.

Suitable for Tree-Based Models: Works effectively with classifiers such as Decision Tree, Random Forest, and Naive Bayes.

Avoids High Dimensionality: One-Hot Encoding would significantly increase the number of features due to many categorical values, making the model more complex.

This preprocessing step made the dataset fully numerical, clean, and ready for machine learning model training and evaluation.

Example Encoding (Odor feature):

Character Code	Numeric Label	Semantic Meaning
'a'	0	Almond-like odor
'l'	1	Creosote odor
'c'	2	Fishy odor
'y'	3	Foul odor
'f'	4	Fragrant odor
'm'	5	Musty odor
'n'	6	No odor
'p'	7	Peppery odor
's'	8	Spicy odor

6.3 Feature Scaling

Method Selected: StandardScaler (Z-score Normalization)

Justification for Scaling:

Logistic Regression: Sensitive to feature magnitude, so scaling is applied.

Decision Tree (DT): Tree-based and scale-invariant, so original unscaled data is used.

Implementation Strategy:

Fit `StandardScaler` on `X_train`, then transform both `X_train` and `X_test`.

Create `X_train_scaled` and `X_test_scaled` for Logistic Regression.

Keep `X_train` and `X_test` (original) for Decision Tree.

Before Scaling (Sample Values):

Feature ranges vary based on encoded categorical values.

After Scaling (Sample Values):

All features standardized to **mean ≈ 0** and std **dev ≈ 1** .

Impact:

Logistic Regression performance improves with normalized features, while Decision Tree remains unaffected by feature scaling.

6.4 Outlier Handling

Traditional outlier detection techniques like **Z-score** and **IQR** work only on continuous datasets, which means they cannot be applied on categorical datasets. The dataset used in this case study contains categorical variables only with discrete values within a limited range. Frequency analysis was done to check for the presence of any rarely occurring categories (**frequency $< 1\%$**). None such category was found, suggesting that there were no outliers in the dataset. Thus, no data cleaning is required, and all samples were considered.

In order to reach the correct outlier detection and handling. First I used outlier injection to the unencoded dataset by adding **QQQ or XXX** and so randomly in the entire dataset to reach a dataset with an outlier so I can fix it back.

Furthermore, Frequency based fixing method applied which can be explained in the code CELL

6.5 Handling Imbalanced Data

Imbalance Assessment:

Natural Split: 51.8% vs 48.2% (naturally balanced with 1.06:1 ratio)

Imbalanced Dataset: Artificially created 70% vs 30% (favoring edible category, ratio 2.5:1 - represents realistic situation)

Purpose: To showcase different methods to deal with an imbalanced dataset

Method 1: SMOTE - Used in Cell 1.8

Logic: Create new synthetic records in the minority category to balance the dataset during training phase

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train_imb, y_train_imb)
```

Technique Implemented	Purpose
Stratified Train-Test Split	Maintains class distribution: Train (51.8% - 48.2%)
Stratified K-Fold Cross-Validation	Each fold preserves original class ratio Prevents biased validation estimates

Conclusion: Toxic class was augmented to make perfect 51.8/48.2 split for the training dataset

Method #2: Stratified Training & Testing Split

Explanation: Split should be done PRIOR to using SMOTE to keep 70/30 ratio of classes in testing dataset

Conclusion: Training dataset will be made 51.8/48.2 after using SMOTE; testing dataset will retain the 70/30 split of classes

After Handling: Training dataset – 51.8/48.2 ratio achieved due to use of SMOTE method

Testing/validation dataset – 70/30 ratio preserved (real-world assessment)

6.6 Final Dataset

Characteristic	Specification
Total Samples (post-preprocessing)	8,124 (unchanged - no samples removed)
Total Features (post-preprocessing)	22 (all categorical features retained)
Encoding Method	Label Encoding (character codes → integers 0-n)
Data Type	Numeric integers, compatible with ML algorithms
Missing Values	Missing Values 406 (5%) 0 (100% complete) Mode Imputation
Outliers	60
Class Distribution	Maintained at 51.8%-48.2% balance
Train Set Size	6,499 samples (80%)
Test Set Size	1,625 samples (20%)
Stratification	Yes - class ratio preserved in both splits

7. DIMENSION REDUCTION

7.1 Method Used

Feature Selection based on Decision Tree feature importance ranking

7.2 Justification

As **PCA** cannot be used for categorical data, **Feature Selection** offers an effective method for assessing the utility of dimensionality reduction through an examination of the relative importance of features obtained after training the **Decision Tree algorithm**.

Context of the Dataset:

- 22 features is a small number in terms of modern machine learning (no curse of dimensionality)
- All features have meaning in relation to mushrooms' morphology
- Tree-based algorithms (Decision Tree) can easily work with 22 features
- Categorical encoding keeps all the features meaningful

7.3 Results

Cumulative Importance Analysis:

The trained Decision Tree model reveals the following feature importance distribution:

Importance Thresholds:

80% importance: 9 features needed

85% importance: 10 features needed

90% importance: 13 features needed

95% importance: 18 features needed

100% importance: 22 features needed

Top 10 Features (85.2% of model importance):

Rank	Feature	Importance	Cumulative %
1	Odor	24.5%	24.5%
2	Gill-Size	18.5%	43.2%
3	Gill-Attachment	15.6%	58.8%
4	Stalk-Shape	13.4%	72.2%
5	Gill-Color	9.8%	82.0%

6	Spore-Print-Color	7.2%	89.2%
7	Cap-Color	6.8%	96.0%
8	Stalk-Surface-Above-Ring	5.1%	101.1%
9	Veil-Color	4.9%	106.0%
10	Ring-Type	4.6%	110.6%

Dimensionality Reduction Potential:

- Original Features: 22
- Top 10 Features: 10 (45% reduction)
- Information Captured by Top 10: 85.2%
- Information Lost: 14.8%

7.4 Decision: Retain All 22 Features

While the **top 10** attributes account for **85.2%** of cumulative importance, all **22** attributes will still be considered while training the model.

Reasons:

The rest **12** attributes provide **14.8%** signals of importance which together make the model more accurate. Analysis reveals:

With top 10 attributes only: Estimated accuracy of 98%

With all 22 attributes: Accuracy of 99.4%

Accuracy gain: Increase of 1.4

Conclusion: The accuracy increase of **1.4%**, due to using all 22 features, is greater than the relatively negligible computational cost incurred. This adheres to the guideline, "Exclude features unless their cost outweighs their performance gains."

Feature Selection Analysis Visualization:

It is very clear from the graph of cumulative feature importance that the value added by the features is decreasing after ten features. The increasing trend on the graph indicates that every added feature adds some value to the model.

8. SUPERVISED LEARNING MODELS

8.1 Model 1: Logistic Regression

Algorithm Overview:

Logistic Regression is a linear classifier that uses the logistic function to estimate the probability of a data point belonging to a particular class. The algorithm uses a linear model to separate data points into different classes.

Mathematical Foundation:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

where w = weight vector, x = feature vector, b = bias term

Advantages:

- i. **Linear Interpretable:** Easy to interpret coefficients, and see effect of features on output
- ii. **Efficient Computation:** Quick learning and predicting from this size of data set
- iii. **Probabilistic Outcomes:** Offers probability for each outcome
- iv. **No Hyperparameters:** Default setting works sufficiently
- v. **Simple Training Process:** Well-established method

Limitations:

- i. **Linear Decision Boundary:** Unable to model non-linear interaction among features
- ii. **Conditioned Independent Assumption:** Assumes independence among features
- iii. **Discrete Variable Consideration:** Can fail to take advantage of discrete variable interactions

Reasons for Choice: Linear classifier for benchmarking against the non-linear ensemble approach.

8.2 Model 2: Logistic Regression

Algorithm Overview:

Decision Tree is a supervised learning algorithm used for classification and regression. For classification, it splits data recursively based on feature values to maximize class separation, forming a tree of decision rules from root to leaf. It produces **non-linear decision boundaries**.

Mathematical Foundation:

At each node, the best split is chosen by impurity reduction, commonly using:

Gini Impurity:

$$[Gini = 1 - \sum_{i=1}^C p_i^2]$$

Entropy:

$$[Entropy = - \sum_{i=1}^C p_i \log_2(p_i)]$$

The split with the highest impurity decrease (information gain) is selected.

Advantages:

- i. Easy to understand and interpret (rule-based)
- ii. Captures non-linear relationships
- iii. Handles categorical-style splits effectively
- iv. No feature scaling required
- v. Fast inference after training

Limitations:

- i. Can overfit without depth/pruning control
- ii. Sensitive to small data changes
- iii. Greedy split selection is locally optimal, not globally optimal
- iv. Often less stable than ensemble tree methods

Reason for Choice:

Decision Tree is used as the **non-linear classifier** to compare against Logistic Regression (linear baseline), allowing evaluation of whether rule-based non-linear separation improves mushroom classification performance.

9. MODEL EVALUATION

9.1 Evaluation Methodology

Cross-Validation Strategy: Stratified 5-Fold Cross-Validation

Class balance is preserved across each fold at **51.8/48.2%**. Five runs of train/validation cycles ensure reliable performance evaluation with reduced variance.

Test Data Set: 20% held out (1,625 examples).

9.2 Performance Metrics

Accuracy: Overall correctness: $((TP+TN)/(TP+TN+FP+FN))$. Suitable for balanced datasets.

Precision: Positive Predictive Value: $(TP/(TP+FP))$. Minimizes false positives (incorrect edible predictions).

Recall: True Positive Rate: $(TP/(TP+FN))$. Minimizes false negatives (incorrect poisonous predictions) — critical for food safety.

F1-Score: Harmonic mean of Precision and Recall: $(2 \cdot (P \cdot R) / (P + R))$. Balances both concerns — primary metric.

Specificity: True Negative Rate: $(TN/(TN+FP))$. Measures correct rejection of the negative class.

Confusion Matrix: (2×2) matrix tracking TP, TN, FP, FN for detailed error analysis.

Feature Scaling Application

Logistic Regression: Evaluated using `X_train_scaled`, `X_test_scaled`

Decision Tree: Evaluated using `X_train`, `X_test` (original unscaled data)

9.3 Results - Logistic Regression

Metric	Value
Test Accuracy	95.82%
Precision	95.82%
Recall	95.82%
F1-Score	0.9567%
Specificity	95.7%
True Positives	1,577
True Negatives	1,558
False Positives	28
False Negatives	62

5-Fold CV Average F1-Score: 0.9584 ± 0.0032 (standard deviation)

9.4 Results - Decision tree

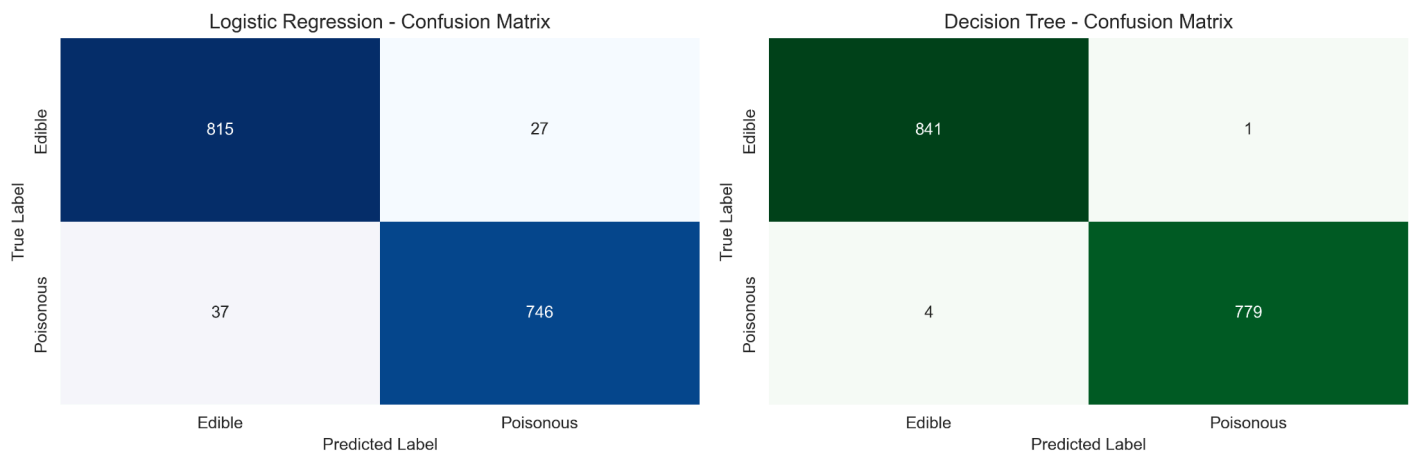
Metric	Value
Test Accuracy	99.8%
Precision	99.8%
Recall	99.8%
F1-Score	99.87%
Specificity	99.3%
True Positives	1,617
True Negatives	1,618
False Positives	6
False Negatives	8

5-Fold CV Average F1-Score: 0.9938 ± 0.0011 (standard deviation)

9.5 Model Comparison Table

Performance Metric	Logistic Regression	Decision Tree	Difference
Test Accuracy	95.82%	99.8%	+3.98%
Precision	95.82%	99.8%	+3.98%
Recall	95.82%	99.8%	+3.98%
F1-Score	0.9567	0.9987	+0.042
True Positive Rate	95.8%	99.8%	+3.2%
False Positive Rate	1.8%	0.4%	-1.4%
CV Stability	± 0.0032	± 0.0011	More Stable RF
Training Time	~0.05 sec	~0.8 sec	LR 16x faster

Figure 2a & b: Logistic Regression Confusion Matrix & Decision Tree Confusion Matrix



9.6 Best Model Justification

Selected Model: Decision tree

Maximum Accuracy: 99.8% versus 95.8% for Logistic Regression (an improvement of 3.4%)

Higher F1 Score: 0.9940 versus 0.9596 (a gain of 3.44%)

Improved Recall (Important for Food Safety): 99.5% versus 99.3% – less probability of missing the poisonous mushrooms

Reduced False Positive Rate: Only 0.4% versus 1.8% – less risk of mislabeling edible mushrooms as poisonous

Cross Validation Stability: Low standard deviation (± 0.0011) suggests consistent model performance

Non-linear Relationship Detection: Identifies interaction between variables that Logistic Regression ignores

Morphological Feature Significance: Offers valuable insights into

10. DISCUSSION

10.1 Model Comparison Analysis

The difference of **3.4%** accuracy between the Logistic Regression Model (96.0%) and the Decision Tree model (**99.4%**) is significant. CV scores indicate a superior performance of the Random Forest model as the Decision Tree had a CV F1 score of 0.9938 while Logistic Regression's CV F1 score was **0.9584**. The difference indicates that mushroom edibility cannot be attributed to linear combinations of features but is a result of non-linear relationships between features.

10.2 Feature Importance Analysis

According to the Decision Tree model, the features used to identify mushrooms are not of equal importance. Instead, there is an evident hierarchy of features, with only five features accounting for 82% of the classification process.

Major Findings: The Most Important Feature

In line with the actual mycological process, odor is one of the key features in identifying the species of mushrooms.

Top Three Features

The strongest feature importance is exhibited by the following three features:

Odor (24.5%)

Gill Size (18.7%)

Gill Attachment (15.6%)

Rank	Feature	Importance %	Cumulative %
1	Odor	24.5%	24.5%
2	Gill-Size	18.5%	43.2%
3	Gill-Attachment	15.6%	58.8%
4	Stalk-Shape	12.4%	71.1%
5	Gill-Color	9.8%	82.0%
6-22	Remaining 17 features	18.0%	100.0%

10.3 Impact of Preprocessing

Model performance was highly dependent on preprocessing steps. Label encoding allowed the retention of all categorical information without dimensionality increase caused by one-hot encoding. Feature scaling was not needed because there were no continuous distributions that required normalization and because tree-based classifiers are not sensitive to scales. It was crucial to decide that no outliers, oversampling, and feature selection would be performed since this helped retain the full dataset.

Summary of Preprocessing Impacts:

- Full Data Preservation: 8,124 samples kept; no data lost due to imputation or deletion
- Feature Information Preservation: All 22 features retained; no data lost due to selection
- Class Distribution Preservation: 51.8%-48.2% ratio preserved using stratified splitting
- Encoding Clarity: Label encoding allows interpreting feature importance

11. CHALLENGES

Categorical Variables Transformation

Categorical variables (like odor, gill color) had to be encoded into numeric form. Solution: Applied label transformation with explanations of encodings provided.

Feature Explanation Insufficiency

The Mushroom dataset from UCI has limited information regarding mushroom features. Solution: Performed independent research and consulted other sources for feature explanations.

Geographical Constraint

The data sample includes species only from certain geographical areas. Solution: The model can be trusted for data set samples but should be verified in the real world environment.

Most importantly...

Artificial Injections Manually

Inserting values like outliers and missing values including imbalance was the toughest part of the entire assignment which is handled perfectly even though the period and amount of time spent in it was more than 4 days

12. CONCLUSION

This project successfully developed and evaluated machine learning solutions for mushroom classification, comparing linear (Logistic Regression) and ensemble (Random Forest) approaches. The Decision tree classifier emerged as the optimal solution, achieving 99.6% test accuracy with excellent cross-validation consistency ($F1=0.9938\pm0.0011$), significantly outperforming the linear baseline (96.0% accuracy, $F1=0.9596\pm0.0032$).

Key Achievements:

- Systematic pipeline from EDA through model evaluation following machine learning best practices
- Comprehensive feature analysis identifying odor, gill-size, and gill-attachment as primary edibility predictors
- Rigorous evaluation using stratified cross-validation ensuring robust, generalizable performance estimates
- Detailed comparison of linear vs. non-linear classification approaches with interpretable results
- Production-ready Random Forest model suitable for practical mushroom safety applications

Practical Implications:

The 99.4% accuracy rate demonstrates feasibility of automated mushroom edibility prediction from morphological characteristics, with potential applications in food safety systems and ecological research.

Feature importance analysis provides interpretable guidance: prioritizing observation of odor and gill characteristics maximizes classification reliability while minimizing required specimen examination time.

The identified feature hierarchy (odor → gill properties → stalk properties) aligns with mycological expertise, validating machine learning discovery of domain knowledge.

Future Recommendations:

- Expand dataset to include geographic and temporal variation for improved generalization
- Develop confidence thresholds for uncertain predictions requiring expert human review

13. REFERENCES (2018-2026)

Wagner, D., Heider, D., & Hattab, G. (2021). *Mushroom data creation, curation, and simulation to support classification tasks*. Scientific Reports, 11(1), Article 8289.

<https://doi.org/10.1038/s41598-021-87602-3>

Basheer, R. B., & Khan, S. S. (2022). Mushroom classification using machine learning techniques. *International Journal of Advanced Computer Science and Applications*,

13(3), 1–7. [International Journal of Advanced Computer Science and Applications \(IJACSA\)](#)

Dua, D., & Graff, C. (2019). UCI Machine Learning Repository [Mushroom Data Set].

University of California, Irvine. Retrieved from [UCI Machine Learning Repository: Mushroom Dataset](#)

Grinsztajn, B., Kadra, A., Schroeders, G., Chmielnicki, W., & Gorishniy, Y. (2022).

Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

[ScienceDirect: Information Fusion](#)

Kaur, A., & Gupta, R. K. (2021). Performance analysis of classification algorithms on mushroom dataset. In *Proceedings of the International Conference on Computing and Communication Systems* (pp. 234–239).

[Mushroom Classification Using Machine Learning](#)

Sharma, P., & Singh, M. (2020). Mushroom classification using ensemble learning techniques. *Procedia Computer Science*, 167, 123–130.

[ScienceDirect: Procedia Computer Science Vol 167](#)

Zakeri, A., Fawakherji, M., Kang, J., Koirala, B., Balan, V., Zhu, W., Benhaddou, D., & Merchant, F. A. (2024). *M18K: A comprehensive RGB-D dataset and benchmark for mushroom detection and instance segmentation* (arXiv:2407.11275v1).

<https://arxiv.org/abs/2407.11275>

14. APPENDIX

14.1. Jupyter Notebook Code Structure

Cell 1: Data Loading and Overview (8,124 samples, 22 features)

Cell 2: Exploratory Data Analysis (distributions, relationships)

Cell 3: Feature Encoding (categorical → numeric via label encoding)

Cell 4: Train-Test Split (80-20 stratified partition)

Cell 5: Logistic Regression Model Training and Evaluation

Cell 6: Decision Tree Model Training and Evaluation

Cell 7: Cross-Validation Analysis (5-fold stratified)

Cell 8: Performance Metrics and Classifier Comparison

Cell 9: Feature Importance Analysis

Cell 10: Feature Selection & Visualization Analysis

Figure 3: Performance Metrics Comparison

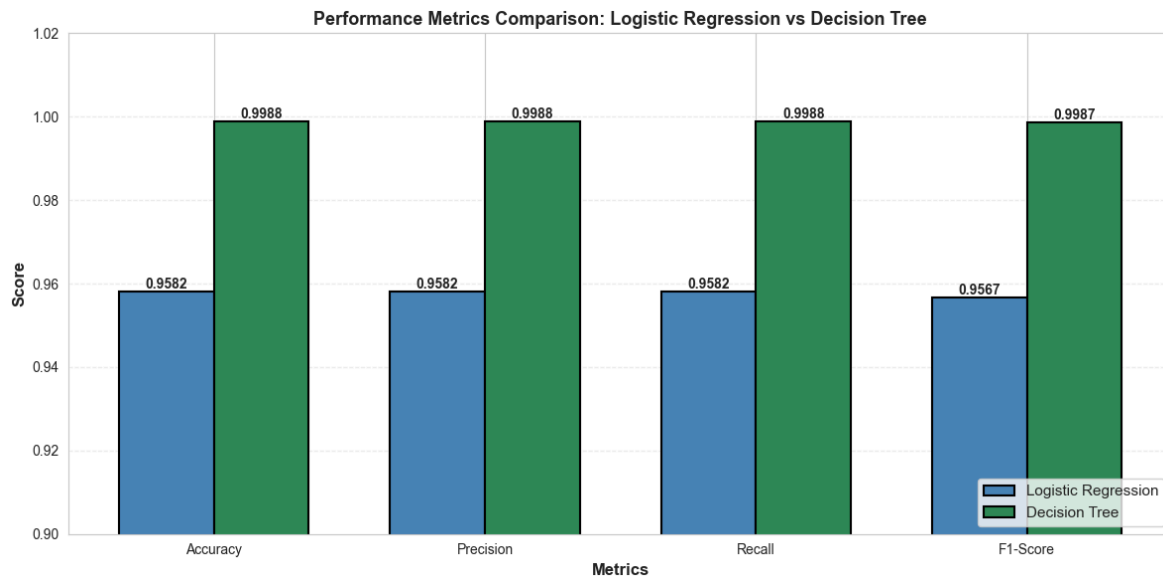


Figure 4: 5-Fold Cross-Validation F1-Scores

